

How the Code Works

UTAS Social Network Analysis System

Prepared 12 August 2026

A walkthrough of the codebase in the order it actually runs: what each layer is responsible for, how a request travels from a click in the browser to a row in the database and back, and where every major feature lives.

1. The Big Picture

The system is **two separate programs** that share no memory and talk only over HTTP. The **backend** is a Python/Flask program that owns the database and performs all the mathematics; it has no visual output and only exchanges JSON. The **frontend** is a React/TypeScript application that runs inside the browser; it stores nothing permanently and simply asks the backend for data and draws it.

That separation is why two servers are started during development, and why the backend can live on PythonAnywhere while the frontend is served from Firebase Hosting. Either side could be replaced without rewriting the other, so long as the JSON contract between them holds.

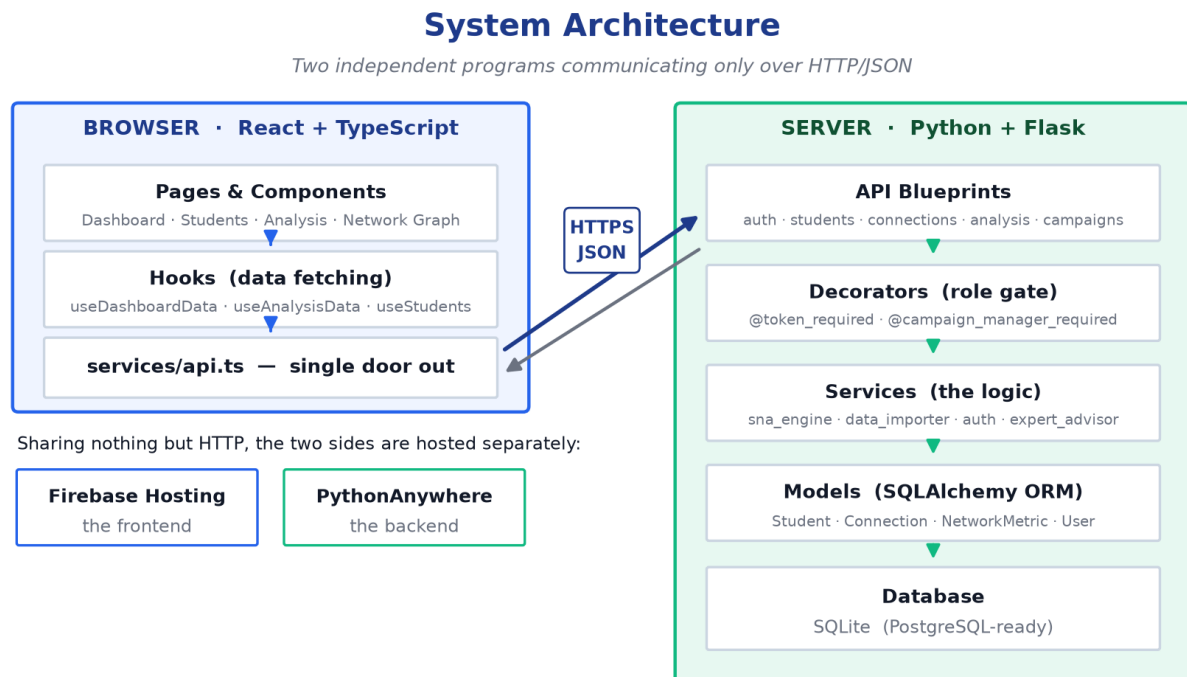


Figure 1: The two halves of the system and the layers inside each. Every request the browser sends carries the user's JWT, which the decorator layer checks before any service runs.

2. The Backend, Layer by Layer

The backend is deliberately organised so that each layer has one job. Reading a feature top-to-bottom means moving through these four folders in order.

2.1 Models — the shape of the data

`app/models/__init__.py` defines seven Python classes that mirror database tables. SQLAlchemy (an Object-Relational Mapper) translates ordinary Python such as `Student.query.filter_by(party='TESCON')` into SQL, so no SQL is written by hand anywhere in the project.

Model	What it stores
Role	The three roles: admin, campaign_manager, student.
User	A login account. Holds a password <i>hash</i> , never the password itself.

Model	What it stores
<code>Student</code>	A person in the network: name, programme, party, tribe, hometown, and so on.
<code>Connection</code>	One friendship. Points at <code>Student</code> twice (from and to) — together, all <code>Connection</code> rows form the edge list of the graph.
<code>NetworkMetric</code>	The computed results for one student: the five centrality scores, community id, influence tier, bridge-node flag.
<code>Campaign</code>	An outreach campaign and the student appointed to manage it.
<code>ActivityLog</code>	An audit trail of actions that changed data.

Deleting a `Student` cascades to that student's `Connection` and `NetworkMetric` rows. Without that cascade, deleting someone left orphaned rows pointing at a student who no longer existed, which crashed the analysis endpoints — a bug this project hit and fixed.

2.2 Services — where the thinking happens

All real logic lives in `app/services/`, kept out of the web layer so it can be tested and reasoned about on its own.

File	Responsibility
<code>sna_engine.py</code>	The heart of the project. Builds the graph and computes every network measure. Explained in section 4.
<code>data_importer.py</code>	Turns a survey spreadsheet into students and connections.
<code>auth_service.py</code>	Registration, login, roles, password changes and resets.
<code>expert_advisor.py</code>	The rule-based advisor behind the Expert Advice button.
<code>report_generator.py</code>	Builds the downloadable PDF report with ReportLab.
<code>activity_service.py</code>	Writes entries to the audit log.

2.3 API — the thin web layer

`app/api/` holds six Flask blueprints — `auth`, `students`, `connections`, `analysis`, `campaigns` and `activity` — registered under `/api/...` in `app/__init__.py`. These files are intentionally thin: read the request, call a service, return JSON. When a route starts growing logic of its own, that is a sign the logic belongs in a service instead.

2.4 Utils — cross-cutting concerns

`jwt_handler.py` issues and refreshes tokens; `decorators.py` guards routes by role; `normalize.py` collapses the many spellings of a programme name into one canonical form.

3. How Security Actually Works

On successful login the server creates a **JSON Web Token** — a cryptographically signed string containing the user's id and role. The browser stores it and attaches it to every later request. Because the signature would break if the contents were altered, a user cannot edit their own token to promote themselves to administrator.

Three decorators enforce access, in `utils/decorators.py`:

```
@token_required          # any logged-in user
@campaign_manager_required # campaign_manager or admin, else 403
@admin_required          # admin only, else 403
```

The point worth emphasising in a defense: **this is enforced on the server, not by hiding buttons in the interface**. A student who discovers the URL of the delete-student endpoint and calls it directly still receives a 403, because the decorator inspects the signed token before the route function is ever reached. Hiding a button is presentation; the decorator is the actual control.

4. The SNA Engine — the core of the project

`services/sna_engine.py` is what makes this a network-analysis system rather than a records-management application. Its `run_full_analysis()` method runs a fixed pipeline:

- **build_graph()** — loads every Student as a node and every Connection as an edge into a NetworkX graph. The graph is *undirected*, because the survey records mutual friendship rather than one-way ties.
- **Five centrality measures** — degree (how many friends), betweenness (who brokers between groups), closeness (who reaches everyone fastest), eigenvector (influence weighted by the influence of one's friends) and PageRank.
- **detect_communities_louvain()** — finds clusters of students nobody labelled in advance, using the Louvain modularity method.
- **identify_bridge_nodes()** — the students whose removal would split the network into disconnected pieces.
- **calculate_influence_tier()** — sorts students into High, Medium and Low.

One design decision is worth defending explicitly. Influence tiers are assigned **by rank** — the top 10 per cent are High, the next 20 per cent Medium — not by fixed score thresholds. Centrality values shrink as a network grows, so on a real sparse network of a few hundred students even the most connected person rarely clears an absolute cut-off such as 0.5. Fixed thresholds silently labelled every student Low; ranking students against each other stays meaningful at any network size.

Finally `save_metrics_to_db()` writes one NetworkMetric row per student. **Everything the interface displays afterwards is read from that table** — the graph is not recomputed when a page loads. That is precisely why the Run Analysis button exists, and why it must be pressed after importing new data.

5. The Frontend, Layer by Layer

`main.tsx` starts the application, `App.tsx` declares the routes, `Layout.tsx` wraps every page in the sidebar and navigation bar, and the pages render inside it.

File	Responsibility
<code>ProtectedRoute.tsx</code>	Redirects to <code>/login</code> when no valid token is present.
<code>context/AuthContext.tsx</code>	Holds who is logged in, so any component can ask without passing props down through every level.
<code>hooks/use*.ts</code>	Data fetching. A page writes <code>useAnalysisData()</code> and receives the numbers, without caring how they were retrieved — which is why the Dashboard and the Analysis page can share data without duplicated code.
<code>services/api.ts</code>	The single door to the backend. Explained below.
<code>components/ui/</code>	Reusable building blocks: Button, Card, Modal, Badge, Pagination.

Every network call passes through one `axios` instance in `api.ts`, which carries two interceptors. The first attaches the token to each outgoing request. The second watches for a 401 reply, silently obtains a fresh token using the refresh token, and retries the original request — which is why a session does not drop the moment the 30-minute access token expires.

6. One Complete Round Trip: Logging In

Following a single action end to end shows how the layers connect.

- The form in `Login.tsx` is submitted, calling `AuthContext.login()`, which calls `api.login()`, which sends `POST /api/auth/login`.
- Flask routes that to `login()` in `api/auth.py`, which calls `AuthService.authenticate_user()`.
- That service hashes the submitted password and compares it with the stored hash. The original password is never stored and cannot be recovered — the server never learns it either.
- On success `JWTHandler.create_tokens()` returns an access token and a refresh token as JSON.
- The browser saves them, `AuthContext` records the user, and React Router navigates to `/dashboard`.
- The Dashboard mounts, its hooks fire several requests, and each one is checked by `@token_required` before any data is returned.

7. The Main Workflow: Survey to Insight

Survey Response to Campaign

path every piece of data takes through the system

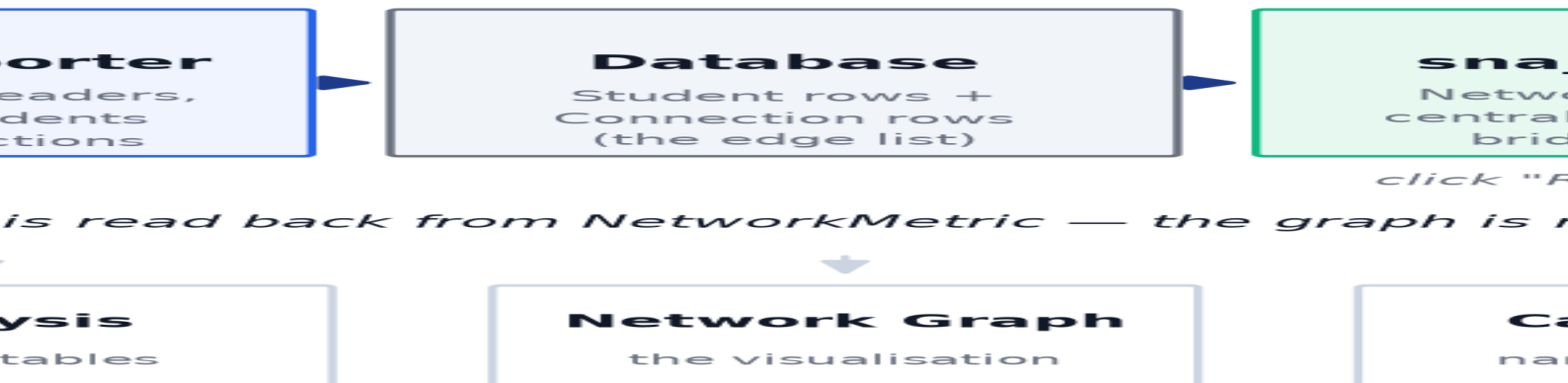


Figure 2: The path from a single survey response to campaign advice.

Import. The Google Forms export is uploaded on the Analysis page. `data_importer.py` normalises the column headers, detects the survey layout, and for each row finds or creates both students before linking them. Students are matched by name, because respondents cannot reliably supply a friend's official student id. This is the least elegant code in the project, and deliberately so: real survey data arrives with reworded questions, curly apostrophes, numeric year cells, refusals typed into name boxes and mutual nominations, and each of those is handled explicitly.

Analyse. Run Analysis invokes the engine described in section 4 and repopulates the NetworkMetric table.

Read. The Dashboard shows headline figures, the Analysis page shows ranked tables per metric, and `NetworkGraph.tsx` draws the force-directed graph, sizing each node by PageRank and colouring it by whichever attribute is selected.

Act. The Campaigns page converts the rankings into named outreach lists, and Expert Advice interprets the same numbers through explicit rules.

8. Tests

backend/tests/ contains 64 automated tests. `conftest.py` gives each test a **fresh in-memory database**, so the suite can never touch or corrupt the real data. The tests exercise the real HTTP endpoints rather than calling functions directly, which means they also verify routing and role permissions, not just isolated logic.

Test file	Covers
<code>test_auth.py</code>	Registration, login, roles, password rules, admin reset, deactivation.
<code>test_students.py</code>	Student CRUD, duplicate detection, cascade deletion.
<code>test_connections.py</code>	Friendship rules: duplicates in both directions, strength validation.
<code>test_campaigns.py</code>	Campaign CRUD, party validation, recommendations.
<code>test_analysis.py</code>	That the analysis endpoints survive orphaned metric rows.
<code>test_data_importer.py</code>	The survey importer, including a full .xlsx upload end to end.
<code>test_normalize.py</code>	Department-name normalisation, including B.Ed versus BSc.

9. Running and Deploying

`run.py` is the development entry point and starts Flask's own server with auto-reload. `wsgi.py` is the production entry point used by the host. `app/__init__.py` chooses SQLite or PostgreSQL from environment variables, so moving to a different database is configuration rather than a rewrite.

```
cd backend
python run.py          # API on port 5000

cd frontend
npm run dev            # interface on port 3000
```

In production the frontend is built to static files and served by Firebase Hosting, while the backend runs on PythonAnywhere. Both use free tiers, so the deployed system carries no running cost.

Where to look first

To change...	Open
A network measure or the analysis pipeline	backend/app/services/sna_engine.py
How survey files are read	backend/app/services/data_importer.py
Who may call an endpoint	backend/app/utils/decorators.py
A database field	backend/app/models/__init__.py
What a page displays	frontend/src/pages/
How data is fetched	frontend/src/hooks/
The graph visualisation	frontend/src/components/Analysis/NetworkGraph.tsx